Développement Mobile : FLUTTER

Résumé des chapitres : Introduction · Les bases (1) · Les bases (2)

Cours & Travaux Pratiques – Cycle Ingénieur – Université de Tunis El Manar, Faculté des Sciences de Tunis – Année 2026-2027

Sommaire

- **Chapitre 1** – Introduction à Flutter : SDK vs Framework, Dart, installation et configuration
- **Chapitre 2** – Les bases (1) : création de projet, structure, main.dart, widgets de base
- **Chapitre 3** – Les bases (2) : listes, formulaires, navigation

Chapitre 1 – Introduction à Flutter

SDK vs Framework

	SDK (Software Development Kit)	Framework
Définition	Boîte à outils complète	Ensemble de briques logicielles réutilisables
Contenu	Compilateur, debugger, bibliothèques ; fournit tout pour développer	Impose une structure au code ; composants prêts à l'emploi
Exemples	Android SDK, iOS SDK, Flutter SDK	Spring, React Native, Flutter Framework

Analogie : SDK = boîte à outils du constructeur ; Framework = plan de l'architecte + briques préfabriquées.

Flutter : un framework multiplateforme

- **Une seule base de code** pour plusieurs plateformes : Android, iOS, Web, Windows, macOS, Linux.
- Basé sur le langage **Dart** : langage moderne développé par Google, syntaxe proche de Java / C# / JavaScript, optimisé pour des interfaces utilisateur réactives.

Installation du Flutter SDK

- **1. Télécharger** : sur flutter.dev, télécharger l'archive ZIP et l'extraire dans un dossier (ex. `C:\src\flutter`).
- **2. Ajouter Flutter au PATH** :

Système	Procédure
Windows	Paramètres système avancés → Variables d'environnement → ajouter `C:\src\flutter\bin` à la variable PATH.
Linux	`nano ~/.bashrc` (ou ~/.zshrc), ajouter `export PATH="$PATH:$HOME/flutter/bin"`, puis `source ~/.bashrc`.
macOS	`nano ~/.zshrc` (ou ~/.bashrc), ajouter l'export du PATH vers `flutter/bin`, puis `source ~/.zshrc`.

- **3. Vérifier** : lancer `flutter doctor` ; Flutter analyse l'environnement et génère un rapport.

Configuration de l'éditeur et des émulateurs

- **Android Studio** : installer, puis File > Settings > Plugins pour ajouter les plugins *Flutter* et *Dart*.
- **VS Code** (alternative) : installer les extensions *Flutter* et *Dart*.
- **Android SDK** : vérifier que Android SDK, Platform-Tools et Build-Tools sont installés.
- **Émulateur Android** : Tools > AVD Manager > Create Virtual Device, choisir un appareil et une image système, Finish, puis lancer avec l'icône de lecture.
- **Appareil réel** : activer le « mode développeur » et le « débogage USB », puis connecter l'appareil par câble USB.
- **Simulateur iOS** (macOS) : installer Xcode via l'App Store, télécharger un simulateur dans Preferences > Components, puis le lancer via Xcode > Open Developer Tool > Simulator (ou depuis VS Code / Android Studio).

Chapitre 2 – Les bases de Flutter (1)

Création d'un nouveau projet

- **Terminal** : se placer dans le répertoire voulu puis exécuter `flutter create mon_premier_projet`.
- **IDE** (VS Code / Android Studio) : File > New Flutter Project, vérifier l'emplacement du SDK, nommer le projet puis cliquer sur Create.

Structure d'un projet Flutter

Élément	Rôle
.idea/ ou .vscode/	Fichiers de configuration de l'IDE
android/	Projet Android natif
ios/	Projet iOS natif (Xcode)
lib/	**Cœur de l'application** Flutter
main.dart	Fichier principal de l'application Dart
test/	Fichiers de test
web/	Fichiers pour le déploiement web
windows/, macos/, linux/	Projets natifs pour applications de bureau
.gitignore	Fichiers à ignorer par Git
pubspec.yaml	Nom, version, description ; gère les **dépendances** (packages) et référence les **assets** (images, polices)
pubspec.lock	Généré automatiquement : versions exactes des dépendances
README.md	Description du projet

Le fichier main.dart

- `import 'package:flutter/material.dart';` : donne accès aux widgets Material Design.
- `void main() { runApp(const MyApp()); }` : point de départ ; `runApp` lance l'application.
- `build(BuildContext context)` : cœur de tout widget, décrit son interface ; *context* donne la position du widget dans l'arbre.
- **MaterialApp** : widget racine qui configure le style Material Design.
- **Scaffold** : structure visuelle de base d'un écran (barre d'application, corps, bouton d'action flottant).
- **AppBar** : barre en haut de l'écran ; **body** : contenu principal.

Introduction aux widgets

Un **widget** est une classe qui décrit une partie de l'interface : **tout est widget** (texte, bouton, espacement, alignement, jusqu'à la page entière). Une application Flutter est un **arbre de widgets** où les parents contiennent des enfants.

Widgets visuels (visibles)	Widgets structuraux (invisibles)
Text, Image, Icon, Bouton, TextField…	Container, Column, Row…

Exemple : une Column contient un Container (texte), une Row avec deux widgets (A et B), puis un autre Container en bas.

Widgets de base

Widget	Description et propriétés clés
Text	Affiche du texte. `style` (TextStyle : fontSize, fontWeight, color, fontStyle), `textAlign`, `maxLines`, `overflow`.
Icon	Icône Material Design : `Icon(Icons.star)`. Propriétés : `color`, `size`.
Container	« Boîte » polyvalente : `width/height`, `padding` (interne), `margin` (externe), `decoration` (BoxDecoration : couleur, bordures, ombres, borderRadius), `alignment`.
Row	Enfants disposés **horizontalement**. `mainAxisAlignment` (axe horizontal) et `crossAxisAlignment` (axe vertical).
Column	Enfants disposés **verticalement** ; mêmes propriétés que Row, axes inversés.

Valeurs d'alignement : mainAxisAlignment = start, center, end, spaceAround, spaceBetween, spaceEvenly ; crossAxisAlignment = start, center, end, stretch, baseline.

```
Container(
  width: 200, height: 100,
  padding: const EdgeInsets.all(10),
  margin: const EdgeInsets.all(20),
  decoration: BoxDecoration(
    color: Colors.green,
    borderRadius: BorderRadius.circular(15)),
  child: Text('Un beau Container'),
)

Row(
  mainAxisAlignment: MainAxisAlignment.spaceEvenly,
  children: [
    Container(color: Colors.red,  width: 150, height: 150, child: Text('case 1')),
    Container(color: Colors.blue, width: 150, height: 150, child: Text('case 2')),
  ],
)
```

Chapitre 3 – Les bases de Flutter (2)

Gestion des listes

- **ListView** : liste défilable. *Statique* avec `children` (liste fixe de ListTile) ou *dynamique* avec `ListView.builder` (`itemCount` + `itemBuilder`), à privilégier pour une liste longue ou de taille inconnue.
- **GridView** : éléments en grille (galeries d'images…). `GridView.count` avec `crossAxisCount` pour fixer le nombre de colonnes.

```
ListView.builder(
  itemCount: items.length,
  itemBuilder: (context, index) {
    return ListTile(title: Text('Item ${items[index]}'));
  },
)

GridView.count(
  crossAxisCount: 3,
  children: <Widget>[ Container(color: Colors.red), /* ... */ ],
)
```

Formulaires

TextField – personnalisation via `InputDecoration` :

- `labelText` (libellé flottant), `hintText` (texte d'exemple), `border` (OutlineInputBorder, UnderlineInputBorder), `icon / prefixIcon / suffixIcon`.
- `keyboardType` : TextInputType.text, .emailAddress, .number, .datetime…
- `obscureText: true` pour masquer le texte (mot de passe).
- Récupération de la saisie avec un **TextEditingController** (`controller:`, puis `.text`).

TextFormField – version de TextField pour les formulaires, avec capacité de **validation** (`validator`). Il s'utilise dans un widget **Form** associé à une clé globale `GlobalKey<FormState>`. La validation se déclenche avec `_formKey.currentState!.validate()`.

```
final _formKey = GlobalKey<FormState>();

Form(key: _formKey, child: Column(children: [
  TextFormField(
    controller: _emailController,
    validator: (value) {
      if (value == null || value.isEmpty) return 'Veuillez entrer votre email';
      if (!value.contains('@')) return 'Email invalide';
      return null; // tout est bon
    },
  ),
  ElevatedButton(
    onPressed: () {
      if (_formKey.currentState!.validate()) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text('Formulaire soumis avec succès!')));
      }
    },
    child: Text('Soumettre'),
  ),
]))
```

Autres champs de formulaire :

Widget	Principe
Checkbox / CheckboxListTile	Valeur booléenne (`value`), mise à jour dans `onChanged` avec `setState` (ex. « J'accepte les termes et conditions »).
Radio / RadioListTile	Choix unique parmi plusieurs (souvent un `enum`, ex. Gender { male, female }) : `value`, `groupValue` (valeur sélectionnée), `onChanged`.
DropdownButton / DropdownMenuItem	Liste déroulante : `value`, `hint`, `items` générés par `.map(...).toList()`, `onChanged`.

Navigation

Méthode	Rôle
`Navigator.push`	Empile une route pour afficher une nouvelle page : `Navigator.push(context, MaterialPageRoute(builder: (context) => SecondPage()))`.
Passage de données (aller)	Via le constructeur de la page cible : `SecondPage(data: 'Hello from the first page')`.
Réception de données	La page cible déclare `final String data;` et un constructeur `SecondPage({required this.data})`, puis affiche `Text(data)`.
`Navigator.pop`	Dépile la route actuelle et retourne à l'écran précédent : `Navigator.pop(context)`.
Retour de données	`Navigator.pop(context, 'valeur')` ; la 1re page récupère le résultat avec `final result = await Navigator.push(...)` dans un `onPressed: ()` `async`. Utile pour formulaires ou sélection dans une liste.

À retenir : Flutter = framework multiplateforme basé sur Dart ; tout est widget et l'UI est un arbre de widgets ; *Container / Row / Column* structurent l'écran, *ListView / GridView* affichent des collections, *Form + TextFormField* valident les saisies, et *Navigator.push / pop* gèrent la navigation et l'échange de données entre écrans.